

DJ Clownfish - AI-Powered DJ Voting Application

Final Project Documentation

Course: COMP4537 - AI Web Service **Project:** Term Project **Team:** c1prj

Table of Contents

1. [Team Member Contributions](#team-member-contributions) 2. [Application Description](#application-description) 3. [Database Design](#database-design) 4. [API Documentation](#api-documentation) 5. [Architecture Overview](#architecture-overview) 6. [Technology Stack](#technology-stack) 7. [Setup Instructions](#setup-instructions) 8. [Deployment Information](#deployment-information) 9. [Testing Information](#testing-information) 10. [Known Limitations](#known-limitations) 11. [Future Enhancements](#future-enhancements)

Team Member Contributions

Peter **Responsibilities:** AI integration, QR code setup, rounding functionality

Actual Contributions: - Implemented AI agent integration with Digital Ocean AI Agent API - Developed song recommendation generation system using AI - Created rounding logic for voting rounds (round progression, status management) - Implemented QR code generation for voting page access - Developed next round generation algorithm that learns from previous voting patterns - Integrated AI prompts and message handling for music recommendations

Yuho **Responsibilities:** API calls tracking and authentication

Actual Contributions: - Implemented comprehensive API tracking middleware - Developed user API consumption tracking system (20 free calls limit) - Created endpoint statistics tracking for admin dashboard - Built API call logging system with detailed request/response tracking - Implemented JWT-based authentication system - Developed user authentication middleware and token validation - Created admin-only endpoints for API statistics and user management - Implemented API rate limiting system

Joao **Responsibilities:** Routes setup (for frontend), database setup (not yet completed, but assigned to him)

Actual Contributions: - Set up all Express.js routes for frontend integration - Organized route structure (auth, voting, spotify, admin, jukebox, AI) - Created route middleware configuration -

Designed route architecture for public and protected endpoints - Database schema design (pending implementation - currently using in-memory storage) - Frontend route integration and API endpoint mapping

Abhi **Responsibilities:** Sync Spotify (calls and callbacks) and authentication

Actual Contributions: - Implemented Spotify OAuth authentication flow - Created Spotify API integration for track search and retrieval - Developed Spotify OAuth callback handling - Implemented Spotify token management and refresh logic - Created Spotify track information endpoints - Integrated Spotify player embeds in frontend - Developed authentication flow for Spotify account linking

Application Description

DJ Clownfish is an interactive music voting platform that combines real-time user engagement with artificial intelligence to create dynamic music experiences. Users participate in voting rounds where they select their preferred songs from AI-generated recommendations, and the system uses machine learning to generate intelligent recommendations for subsequent rounds.

Key Features

Owner Features - ****Authentication:**** Secure login/signup with JWT tokens - ****Create Voting Rounds:**** Set up voting rounds with criteria like genre, BPM, artists, mood, and energy level - ****QR Code Generation:**** Generate QR codes for participants to easily access voting pages - ****Round Management:**** - Pause/Resume voting rounds - View real-time voting results - Generate next rounds with AI based on voting patterns - ****Spotify Integration:**** View and play winning songs directly via Spotify embed player - ****Dashboard:**** Comprehensive dashboard to manage all voting rounds

Participant Features - ****Easy Access:**** Scan QR code to access voting page instantly - ****Vote on Songs:**** Vote for favorite songs from AI-generated recommendations (one vote per round) - ****Real-time Results:**** See live vote counts updating every 5 seconds - ****Winner Display:**** View current winner with Spotify player integration - ****No Registration Required:**** Participants can vote without creating an account

AI-Powered Recommendations - ****Smart Song Generation:**** AI agent analyzes voting patterns and generates 10 new song recommendations - ****Learning System:**** Each round learns from previous winners and voting preferences - ****Criteria-Based:**** Recommendations consider genre, BPM, artists, mood, and energy level

Database Design

Design Principles

Following database normalization principles, each column serves only one purpose, and all primary keys are unique integer numbers (not codes or meaningful strings).

Schema Design

Users Table

```
CREATE TABLE user (  
  user_id INT(11) PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  creation_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  api_calls INT(11) NOT NULL DEFAULT 0  
);
```

Notes: - `user\_id` is a unique integer primary key (not email or other business identifier) - `email` is stored separately as a unique constraint for business logic - `password` stores the hashed password (hashed with bcrypt before storage) - `api\_calls` tracks the number of API calls made by the user (for the 20 free calls limit) - `creation\_date` automatically records when the user was created

Admin Table

```
CREATE TABLE admin (  
  admin_id INT(11) PRIMARY KEY AUTO_INCREMENT,  
  user_id INT(11) NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES user(user_id)  
);
```

Notes: - `admin\_id` is a unique integer primary key - `user\_id` is a foreign key reference to the `user` table - This table establishes which users have admin privileges - A user becomes an admin by having a record in this table

Database Design Principles Applied

1. ****Single Purpose Per Column****: Each column serves only one purpose - Primary keys are unique integers (not codes or meaningful strings) - Business identifiers (like email) are stored in separate columns - Example: `user\_id` is the PK, `email` is a separate unique column

2. **Normalization**: - Separate tables for different entities (users and admin privileges) - Foreign keys maintain referential integrity - Admin privileges are stored in a separate table linked to users
3. **Naming Conventions**: - Primary keys follow pattern: `{table\_name}\_id` (e.g., `user\_id`, `admin\_id`) - Foreign keys reference the primary key name (e.g., `user\_id` in admin table references `user\_id`)

Current Implementation Status

Note: Currently, the application uses: - **In-memory storage** for all data (users, voting rounds, songs, votes, API tracking) - **Database tables** (`user` and `admin`) exist in the database but are **not currently connected** to the application code - The application code stores all data in memory arrays (data is lost on server restart)

Database Tables (Created Externally, Not Currently Used): - `user` table - Exists in database but application uses in-memory `users` array - `admin` table - Exists in database but application uses in-memory role checking

Future Implementation: The database design follows proper normalization principles with integer primary keys and separate tables for different entities. To connect the application to the database, you would need to: 1. Install a database driver (e.g., `mysql2`, `pg`, `mongodb`) 2. Create database connection configuration 3. Replace in-memory storage with database queries in: - `authService.js` - Replace `users` array with database queries - `votingService.js` - Replace `votingRounds`, `songs`, `votes` arrays with database tables - `apiTrackingMiddleware.js` - Replace in-memory tracking with database logging

API Documentation

Documentation Location

The API documentation is available in two formats:

1. **Interactive Swagger UI**: Available at `/doc/` endpoint - Example: `http://localhost:3000/doc/` - Interactive interface to test endpoints - Complete request/response schemas
2. **Static Documentation**: See `API\_DOCUMENTATION.md` file - Complete endpoint reference - Request/response examples - Authentication requirements

API Overview

The API follows RESTful principles and includes:

- **Authentication Endpoints:** User registration, login, profile management - **AI Agent Endpoints:** Communication with Digital Ocean AI Agent - **Voting Endpoints:** Round creation, voting, results - **Spotify Endpoints:** Track search, OAuth, playback - **Admin Endpoints:** API statistics, user management - **Jukebox Endpoints:** Automated playlist management - **Health Endpoints:** Server status checks

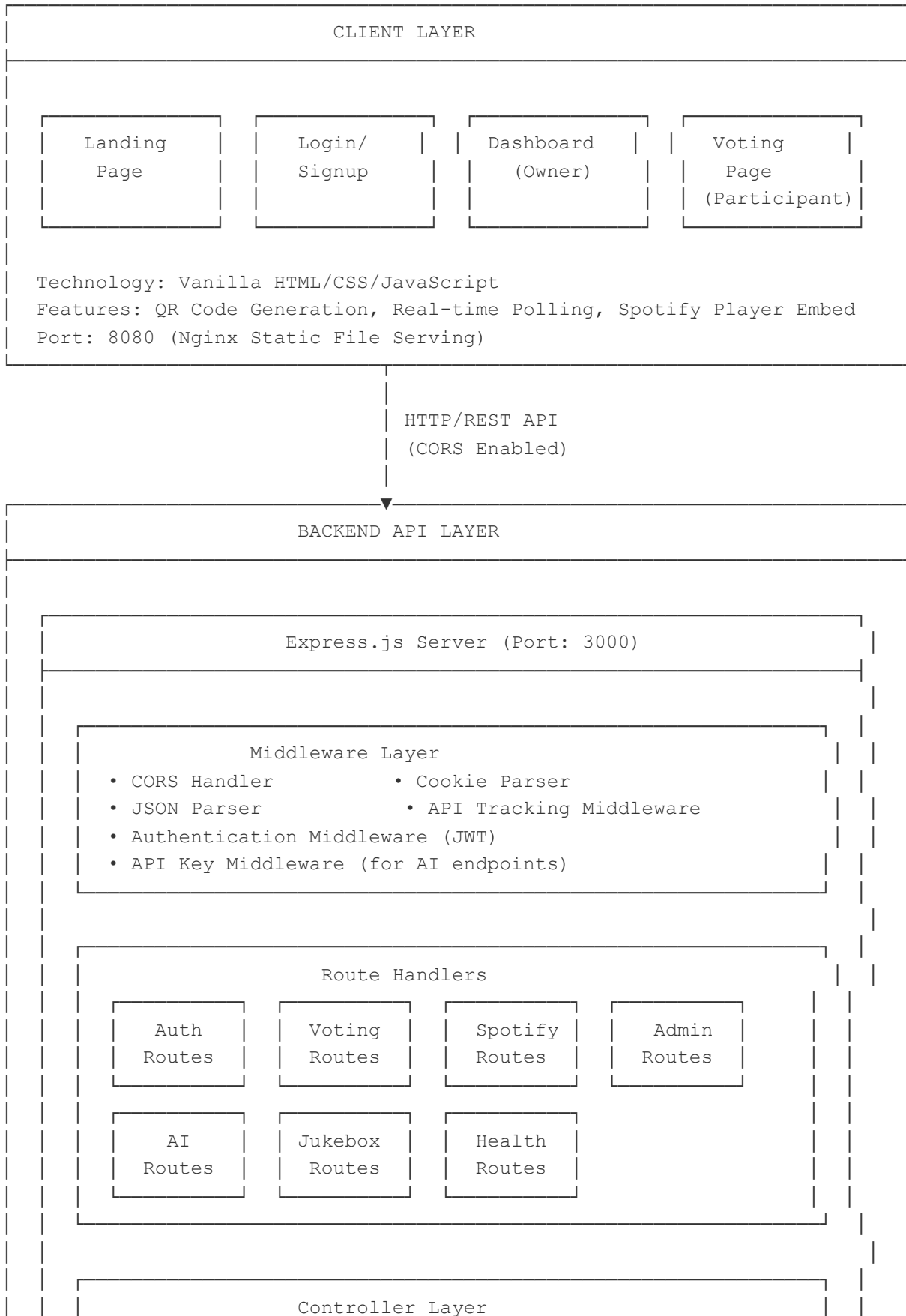
Key API Features

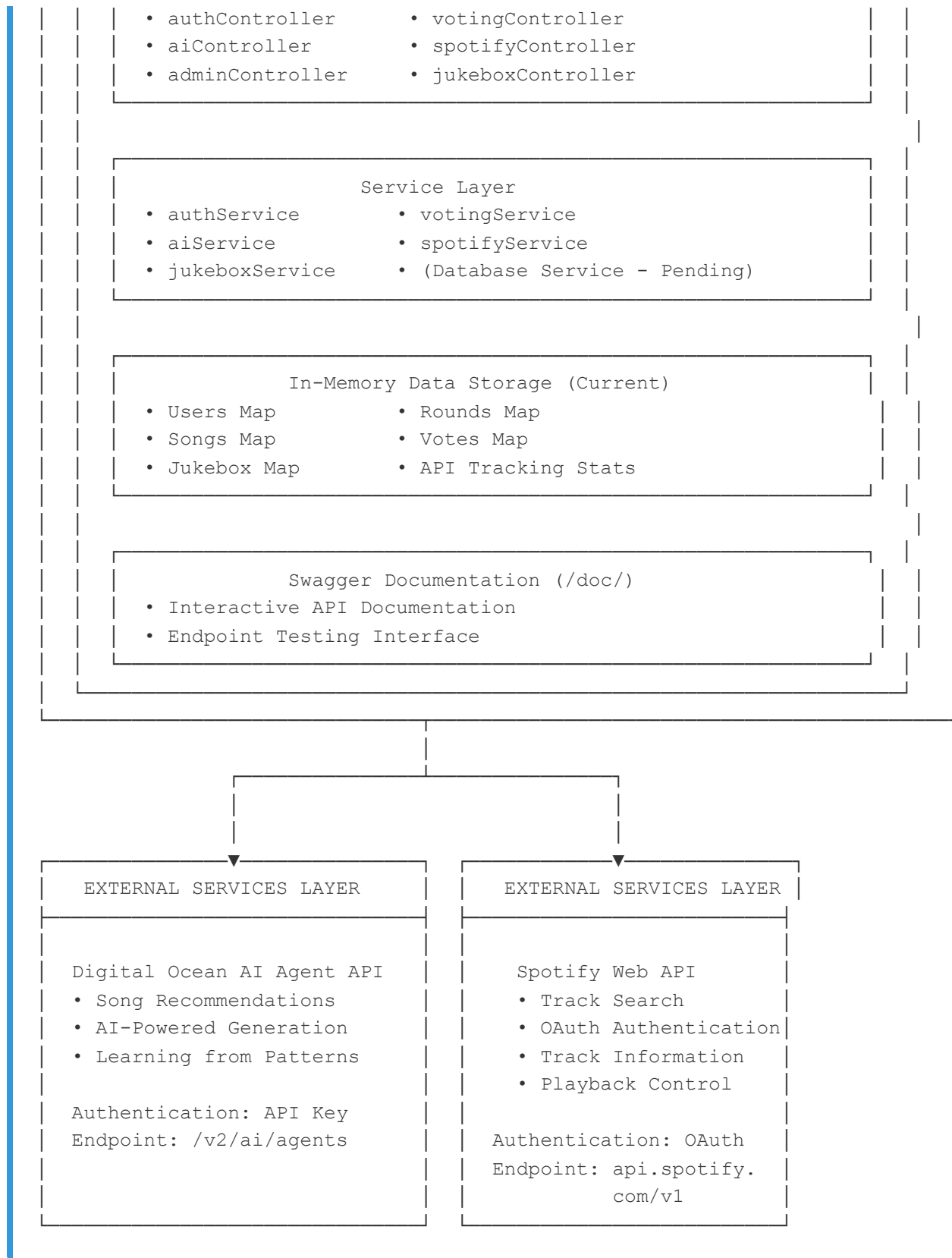
- JWT-based authentication for protected endpoints - API key authentication for AI agent endpoints - Rate limiting (20 free calls per user, unlimited for admins) - Comprehensive error handling - Detailed API tracking and logging

For complete API documentation, refer to: - Swagger UI at `/doc/` endpoint - `API_DOCUMENTATION.md` file

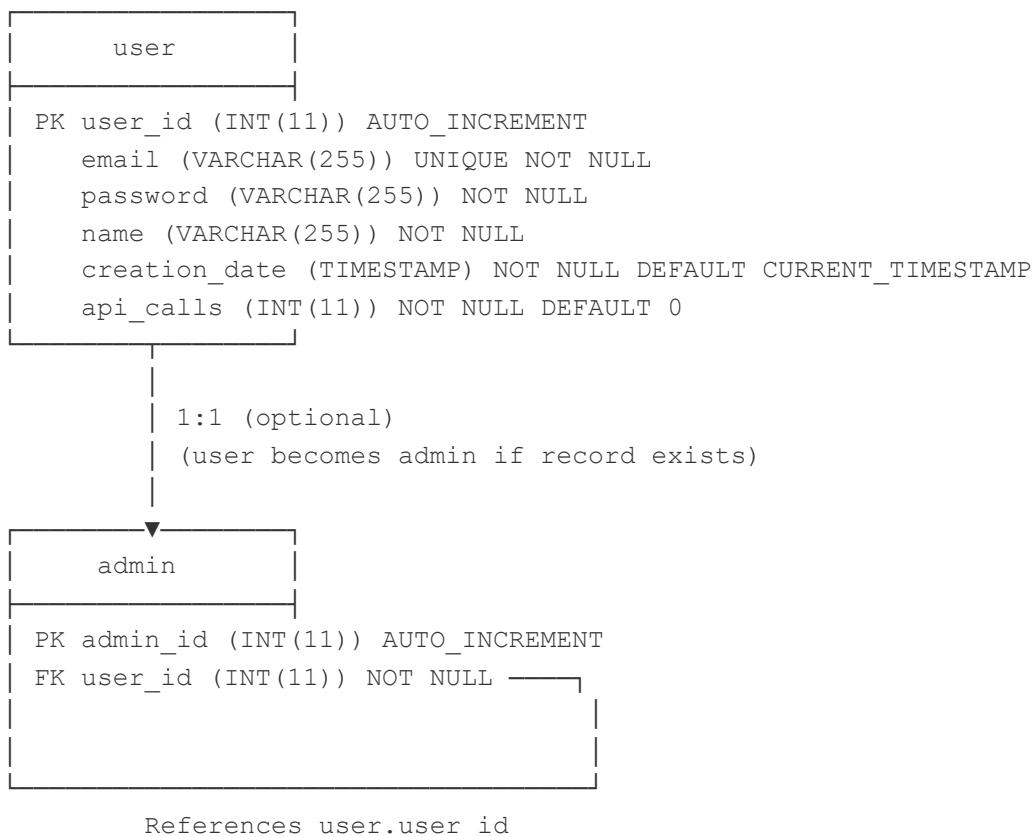
Architecture Overview

Detailed System Architecture Diagram





Database Entity Relationship Diagram (Current Implementation)



Relationship Explanation: - Each user can have at most one admin record (1:1 relationship) - A user becomes an admin by having a corresponding record in the `admin` table - The `user\_id` in the `admin` table is a foreign key that references `user.user\_id` - The `api\_calls` column in the `user` table tracks API consumption (20 free calls limit)

Component Structure

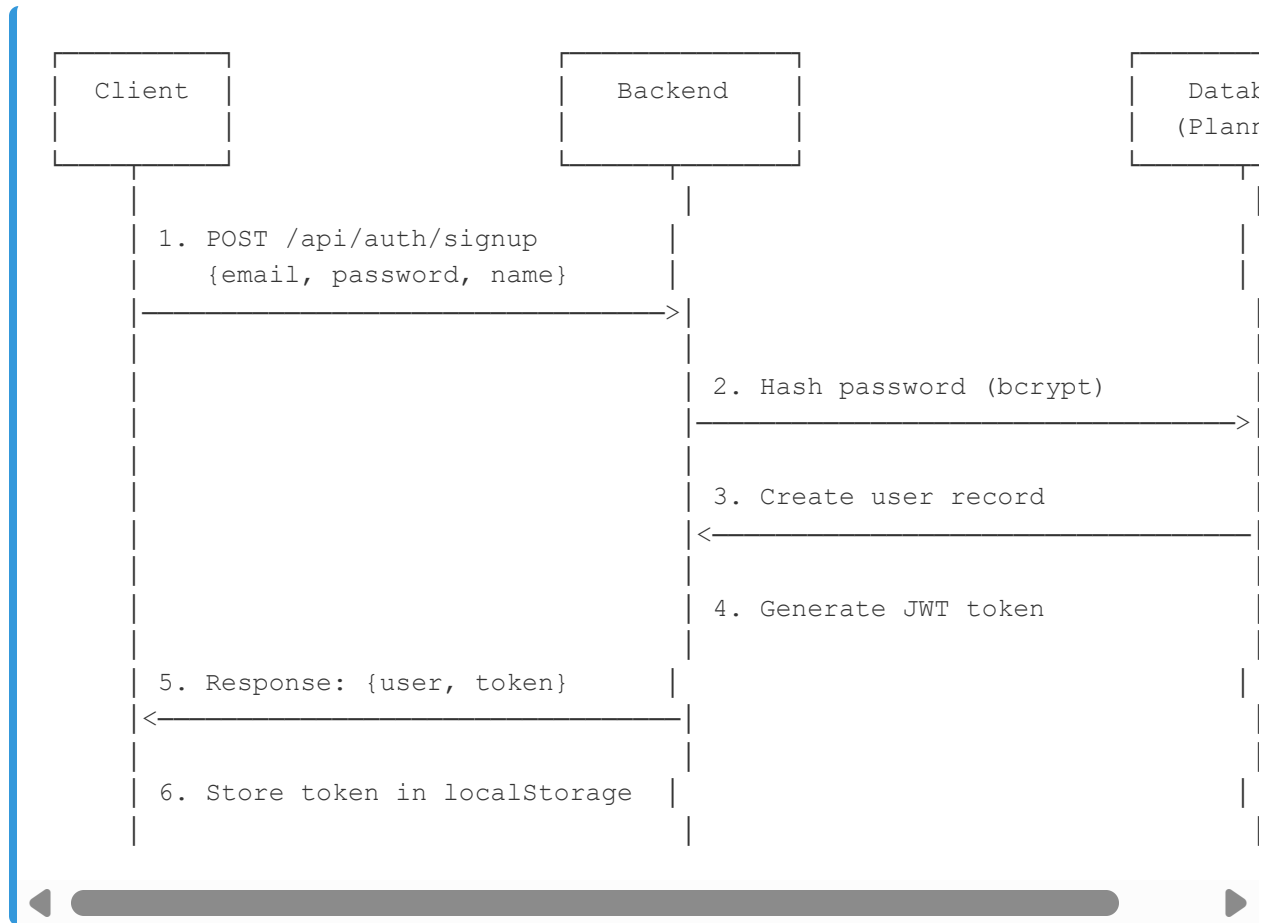
Frontend (Static Site) - **Technology:** Vanilla HTML/CSS/JavaScript - **Hosting:** Nginx static file serving - **Features:** - Responsive design - QR code generation (client-side) - Real-time updates via polling - Spotify player integration

Backend (Web Service) - **Technology:** Node.js with Express.js - **API:** RESTful API with JWT authentication - **Services:** - Authentication service (JWT, bcrypt) - Voting service (in-memory storage) - AI service (Digital Ocean AI Agent integration) - Spotify service (Web API integration) - Jukebox service (automated playlist management) - API tracking middleware

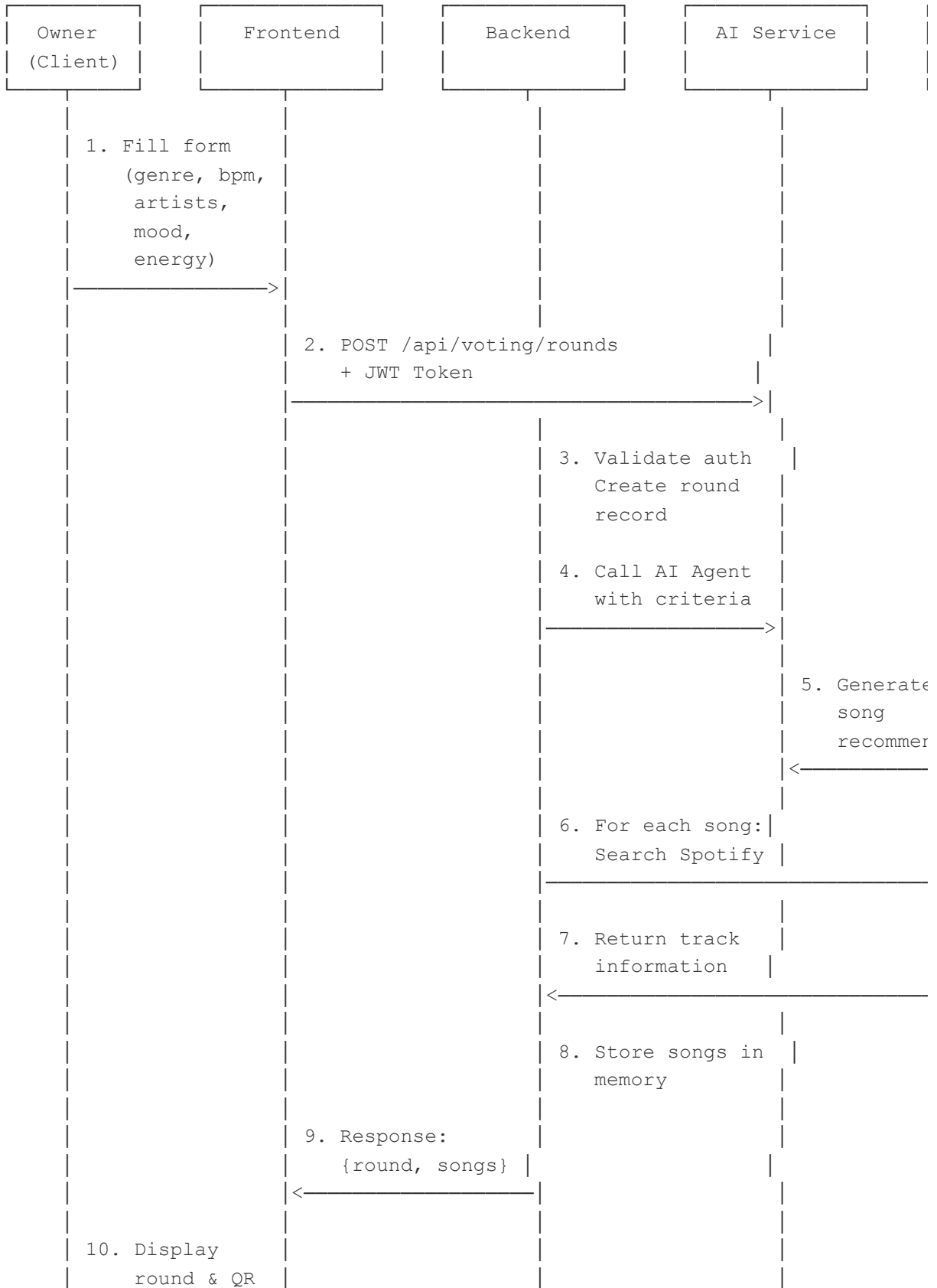
External Services - **Digital Ocean AI Agent:** For intelligent song recommendation generation - **Spotify Web API:** For track search and playback - **QR Code Library:** Client-side QR code generation (qrcode.js CDN)

Detailed Data Flow Diagrams

1. Authentication Flow

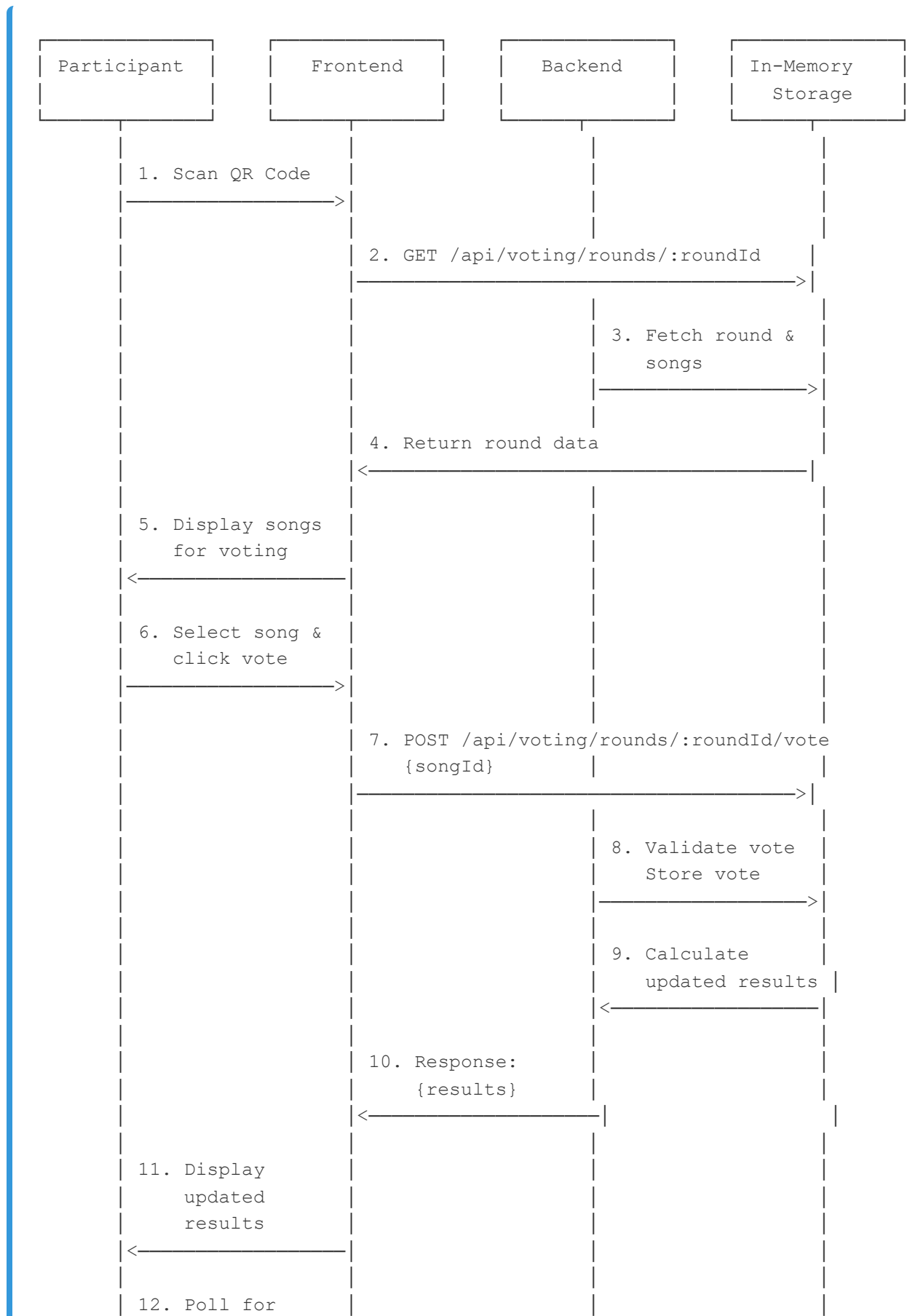


2. Voting Round Creation Flow



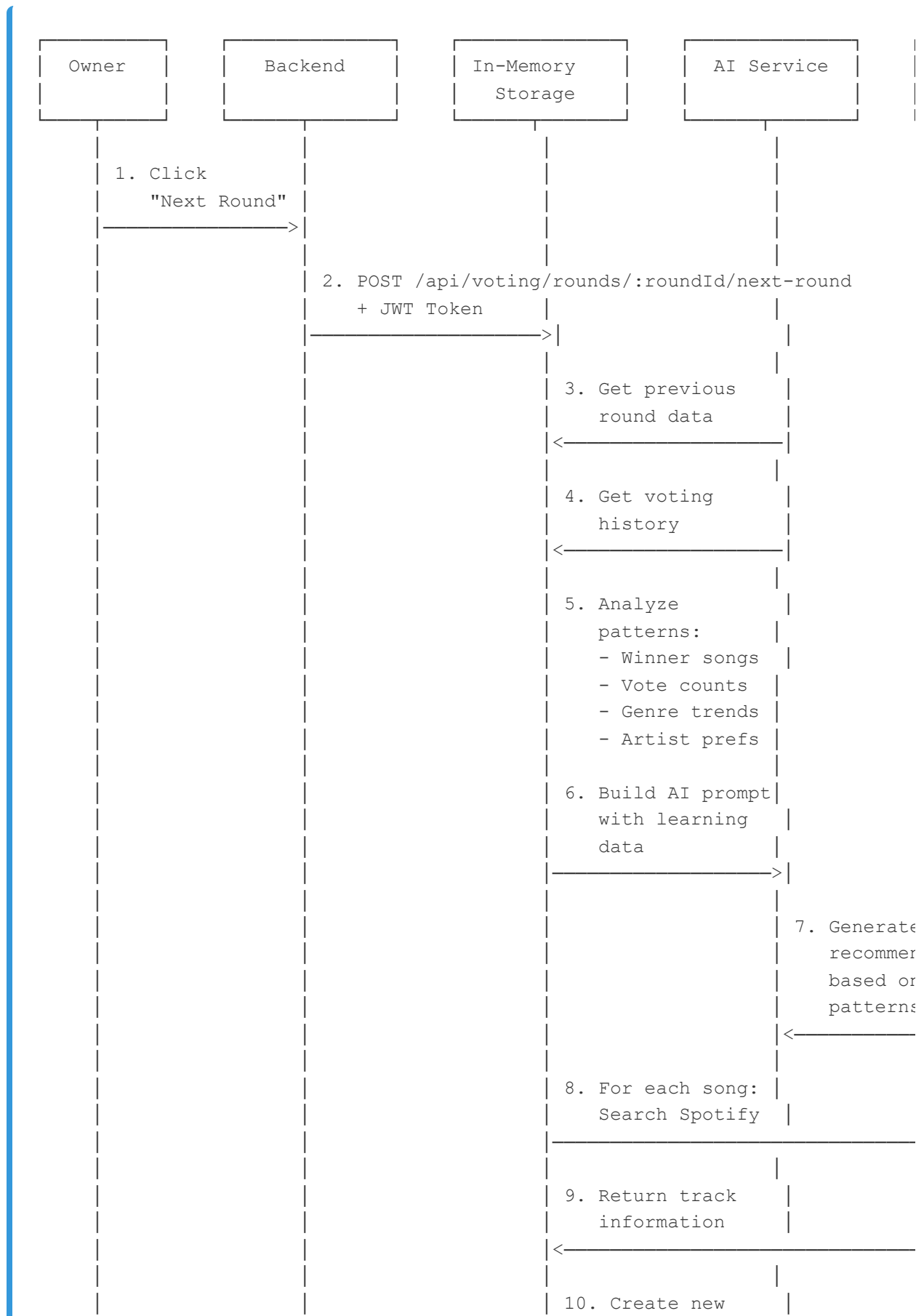


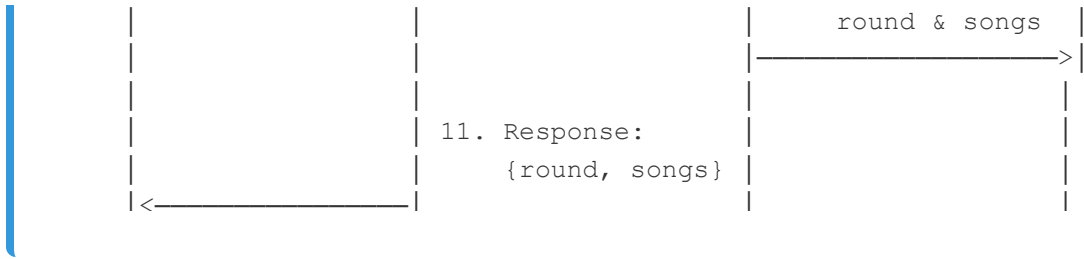
3. Voting Flow (Participant)



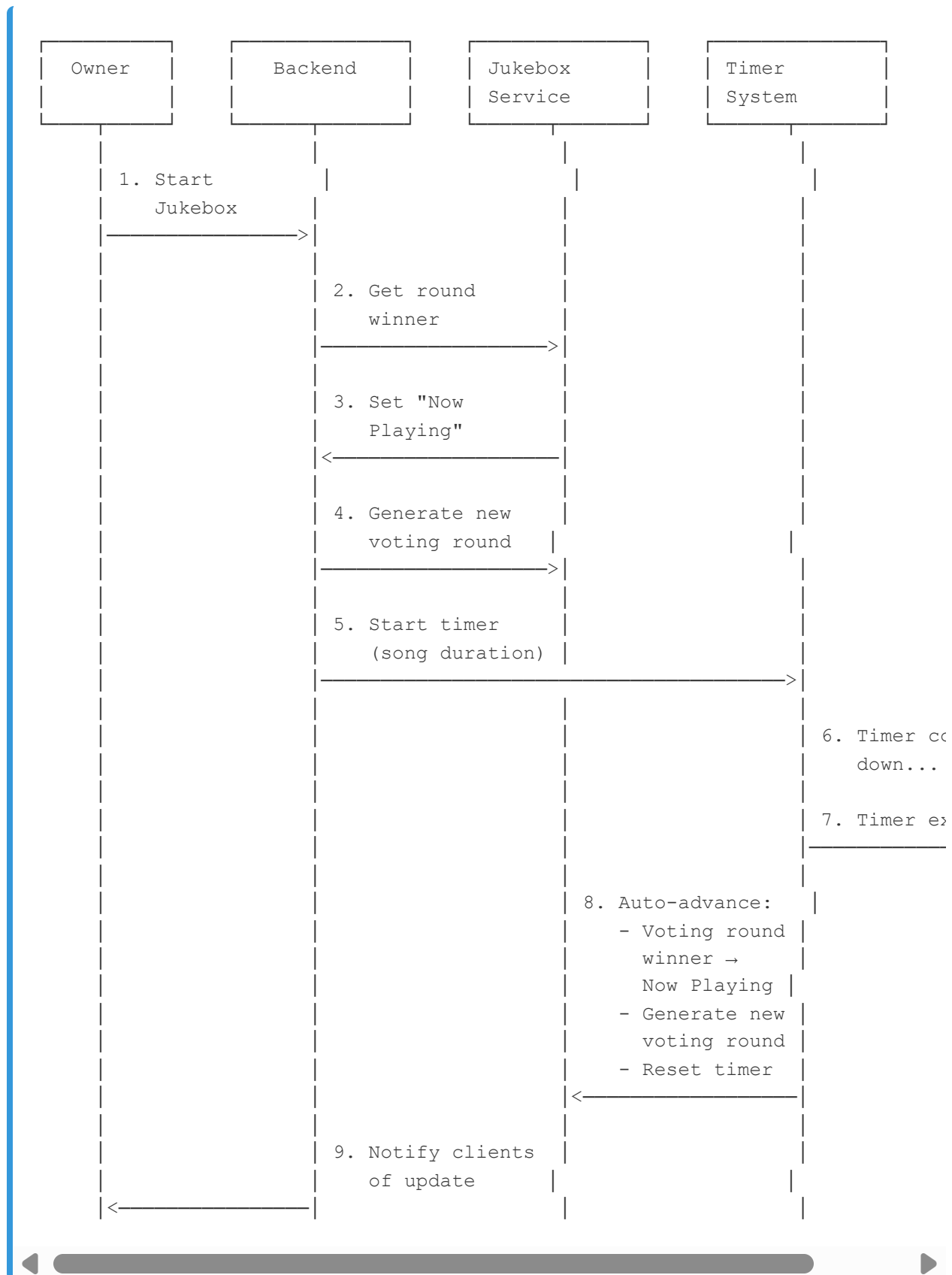


4. Next Round Generation Flow (AI Learning)

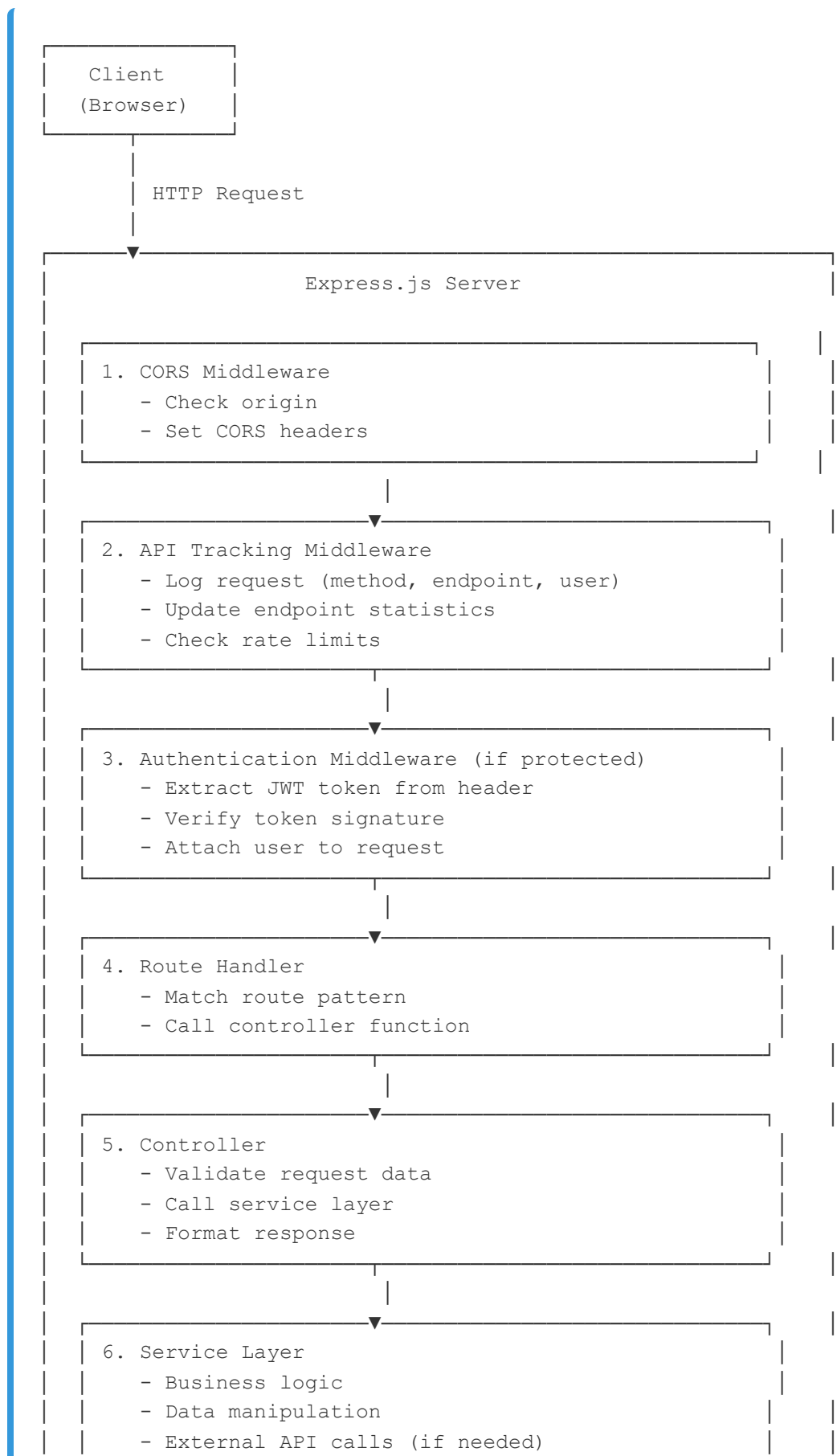


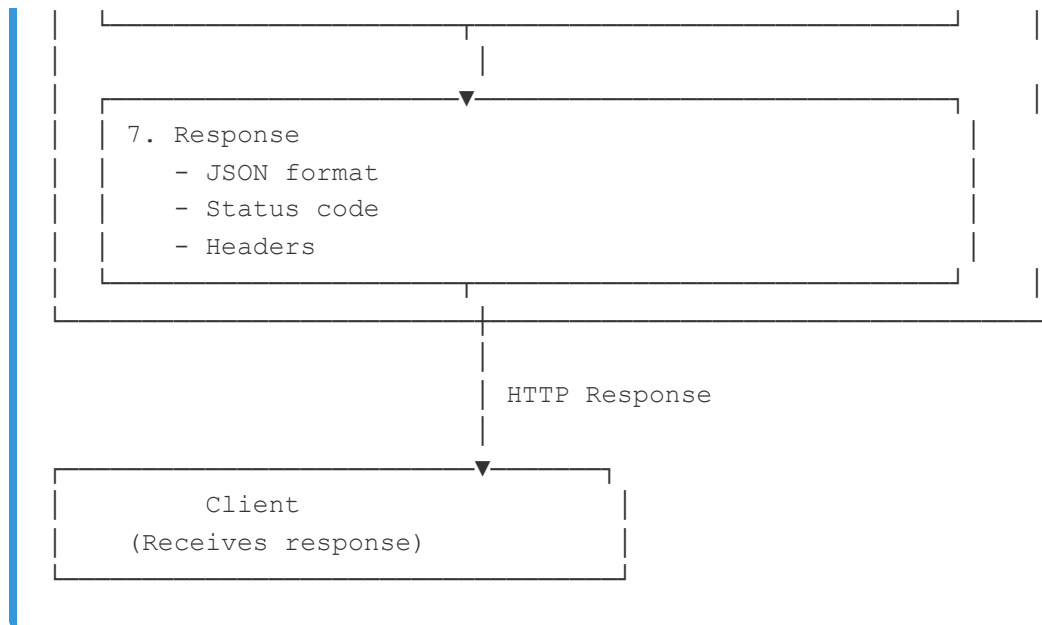


5. Jukebox Mode Flow



API Request Flow Diagram





Component Structure

Technology Stack

Backend - **Runtime:** Node.js (v14+) - **Framework:** Express.js - **Authentication:** JWT (jsonwebtoken), bcrypt - **API Documentation:** Swagger (swagger-jsdoc, swagger-ui-express) - **HTTP Client:** Axios - **Environment:** dotenv

Frontend - **Core:** Vanilla HTML5, CSS3, JavaScript (ES6+) - **QR Code:** qrcode.js (CDN) - **HTTP Client:** Fetch API - **Styling:** Custom CSS with responsive design

External APIs - **AI Agent:** Digital Ocean AI Agent API - **Music:** Spotify Web API - **Authentication:** JWT tokens

Development Tools - **Package Manager:** npm - **Process Manager:** nodemon (development) - **Version Control:** Git

Deployment - **Containerization:** Docker - **Web Server:** Nginx (frontend) - **Process Management:** Node.js (backend)

Setup Instructions

Prerequisites - Node.js (v14 or higher) - npm or yarn - Spotify Developer Account (for Spotify API access) - Digital Ocean AI Agent API key - Nginx (for frontend hosting in production)

Backend Setup

1. **Navigate to backend directory:**

```
cd backend
```

2. **Install dependencies:**

```
npm install
```

3. **Create `.env` file:**

```
PORT=3000
JWT_SECRET=your_jwt_secret_here
AI_AGENT_API_KEY=your_digital_ocean_ai_agent_api_key
AI_AGENT_URL=https://api.digitalocean.com/v2/ai/agents
SPOTIFY_CLIENT_ID=your_spotify_client_id
SPOTIFY_CLIENT_SECRET=your_spotify_client_secret
FRONTEND_URL=http://localhost:8080
```

4. **Start the server:**

```
npm start
# Or for development with auto-reload:
npm run dev
```

Frontend Setup

1. **Navigate to frontend directory:**

```
cd frontend
```

2. **Update backend URL in HTML files:** Edit the following files and update `BACKEND_URL` : -
``public/index.html`` - ``public/login.html`` - ``public/signup.html`` - ``public/dashboard.html`` -
``public/vote.html``

Change:

```
window.BACKEND_URL = 'http://localhost:3000';
```

3. ****For local development, serve static files:****

```
# Using Python
python -m http.server 8080

# Or using Node.js http-server
npm http-server public -p 8080
```

4. ****For production, use Nginx:**** - See `frontend/nginx.conf` for configuration - Deploy using Docker (see `frontend/Dockerfile`)

Docker Deployment

Backend

```
cd backend
docker build -t dj-clownfish-backend .
docker run -p 3000:3000 --env-file .env dj-clownfish-backend
```

Frontend

```
cd frontend
docker build -t dj-clownfish-frontend .
docker run -p 8080:80 dj-clownfish-frontend
```

Verification

1. ****Check backend health:****

```
curl http://localhost:3000/health
```

2. ****Check API documentation:**** Open browser: <http://localhost:3000/doc/>

3. ****Check frontend:**** Open browser: <http://localhost:8080>

Deployment Information

Production Environment

- **Backend:** Deployed on [Your Production Server] - **Frontend:** Deployed on [Your Production Server] - **Domain:** [Your Domain]

Environment Variables

All sensitive configuration is managed through environment variables: - JWT secrets - API keys - Database credentials (when implemented) - CORS allowed origins

Security Considerations

- JWT tokens for authentication - Password hashing with bcrypt - CORS configuration for allowed origins - API rate limiting - Input validation on all endpoints - Secure cookie handling for participant tokens

Testing Information

Manual Testing

1. **Authentication Flow:** - Test user registration - Test user login - Test protected endpoints with/without tokens
2. **Voting Flow:** - Create voting round - Generate QR code - Submit votes - View results - Generate next round
3. **AI Integration:** - Test AI agent health check - Test song generation - Test next round generation
4. **Spotify Integration:** - Test OAuth flow - Test track search - Test track information retrieval

API Testing

Use Swagger UI at </doc/> endpoint for interactive API testing: - Test all endpoints - View request/response formats - Test authentication flows

Known Test Cases

-  User registration and login -  Voting round creation -  AI song generation -  Vote submission -  Results retrieval -  QR code generation -  Spotify integration -  Admin statistics -  Jukebox mode

Known Limitations

Current Limitations

1. **In-Memory Storage:** - All data is stored in memory - Data is lost on server restart - Not suitable for production scale
2. **Single Server:** - Not designed for horizontal scaling - State stored in memory (not shared across instances)
3. **No Database:** - Database implementation is pending - Currently using in-memory data structures
4. **Rate Limiting:** - Simple in-memory rate limiting - Not persistent across restarts
5. **Real-time Updates:** - Uses polling instead of WebSockets - Not true real-time updates

Planned Improvements

- Database integration (assigned to Joao) - Persistent storage for all data - WebSocket support for real-time updates - Horizontal scaling support - Redis for rate limiting - Database-backed session management

Future Enhancements

Short-term Goals

- ☐ Database integration (MongoDB/PostgreSQL) - ☐ Persistent data storage - ☐ User profile management - ☐ Voting history tracking

Medium-term Goals

- ☐ WebSocket support for real-time updates - ☐ Advanced analytics and insights - ☐ Multi-round tournaments - ☐ Social sharing features - ☐ Mobile app (React Native/Flutter)

Long-term Goals

- ☐ Machine learning model training on voting patterns - ☐ Personalized recommendations per user - ☐ Integration with multiple music platforms - ☐ Advanced admin panel - ☐ Multi-language support - ☐ Internationalization

Conclusion

DJ Clownfish is a comprehensive AI-powered music voting platform that successfully integrates multiple technologies to create an engaging user experience. The application demonstrates:

- RESTful API design - AI integration for intelligent recommendations - Real-time voting and results - Secure authentication and authorization - Comprehensive API documentation - Scalable architecture (pending database implementation)

The project successfully meets the requirements for the COMP4537 term project, providing a solid foundation for future enhancements and production deployment.

Appendix

API Documentation - Swagger UI: `/doc/` endpoint - Static Documentation: `API_DOCUMENTATION.md`

Additional Resources - README.md - General project information - API_CALLS_GUIDE.md - API tracking usage guide - JUKEBOX_GUIDE.md - Jukebox mode guide - SETUP_CHECKLIST.md - Setup verification checklist

Contact Information For questions or issues, please contact the development team.

Document Version: 1.0 **Last Updated:** 11/23/2025 **Project Status:** Active Development